



Data Analysis and Visualization

3CS103ME24

Term Assignment

23BEC177 (Aditi Shah)
23BEI020 (Eera Bhargava)

Submitted To
Dr Jai Prakash V. Verma

Objective

The primary objective of this project is to analyse and predict employee attrition by leveraging data analytics and machine learning techniques. The study aims to identify key factors that contribute to employee turnover, such as overtime, work-life balance, salary, promotion history, and job satisfaction. By uncovering these underlying causes, the project seeks to assist organizations in making informed, data-driven decisions to enhance employee retention and well-being. Additionally, the project focuses on building and comparing various predictive models—such as Logistic Regression, Random Forest, SVM, Decision Tree, and K-Nearest Neighbours—to determine which provides the most accurate and reliable predictions. Through the integration of Tableau visualizations and machine learning insights, this project strives to bridge descriptive and predictive analytics, offering HR departments actionable strategies to proactively address workforce challenges.

Scope

The scope of this project encompasses the complete process of data collection, preprocessing, visualization, model development, and insight generation using the IBM HR Analytics dataset. It includes cleaning and encoding data, handling outliers, scaling features, and performing exploratory data analysis to uncover meaningful trends. Tableau dashboards are developed to visualize key metrics such as attrition rate, department-wise turnover, job satisfaction, and work-life balance. The project also applies multiple machine learning models to predict whether an employee is likely to leave, followed by performance evaluation using metrics like accuracy, precision, recall, and F1-score. Beyond prediction, the project interprets results to highlight the most influential features affecting attrition. The outcomes aim to support HR professionals in designing retention strategies, improving workplace satisfaction, and developing a data-driven culture for sustainable human resource management.

Dataset : IBM HR Analytics Employee Attrition & Performance

Kaggle Link : <https://www.kaggle.com/datasets/pavansubhasht/ibm-hr-analytics-attrition-dataset?resource=download>

No. of Rows: 1470 **No.**

of Columns: 35

Columns Description:

- **Age:** Age of the employee (Numerical).
- **Attrition:** Indicates if an employee has left the company ("Yes" or "No") (Categorical).
- **BusinessTravel:** Frequency of business travel (e.g., Travel_Rarely, Travel_Frequently) (Categorical).
- **DailyRate:** Daily wage rate of the employee (Numerical).
- **Department:** Department where the employee works (e.g., Sales, Research & Development) (Categorical).
- **DistanceFromHome:** Distance between the employee's home and workplace (Numerical).
- **Education:** Education level (1 = Below College, 2 = College, etc.) (Ordinal).
- **EducationField:** Field of study (e.g., Life Sciences, Medical) (Categorical).
- **EmployeeCount:** Number of employees (always 1, likely a placeholder) (Numerical, constant).
- **EmployeeNumber:** Unique identifier for each employee (Numerical).
- **EnvironmentSatisfaction:** Job environment satisfaction level (1-4) (Ordinal).
- **Gender:** Gender of the employee (Male/Female) (Categorical).
- **HourlyRate:** Hourly wage rate of the employee (Numerical).
- **JobInvolvement:** Level of involvement in the job (1-4) (Ordinal).
- **JobLevel:** Position level within the organization (1-5) (Ordinal).
- **JobRole:** Designation or role (e.g., Sales Executive, Research Scientist) (Categorical).
- **JobSatisfaction:** Satisfaction level with the job (1-4) (Ordinal).
- **MaritalStatus:** Marital status (Single, Married, Divorced) (Categorical).
- **MonthlyIncome:** Monthly income of the employee (Numerical).
- **MonthlyRate:** Monthly rate or salary structure (Numerical).

- **NumCompaniesWorked:** Total number of companies the employee has worked for (Numerical).
- **Over18:** Indicates if the employee is above 18 years (Y) (Categorical, constant).
- **OverTime:** Whether the employee works overtime (Yes/No) (Categorical).
- **PercentSalaryHike:** Percentage increase in salary (Numerical).
- **PerformanceRating:** Performance rating (1-4) (Ordinal).
- **RelationshipSatisfaction:** Satisfaction with relationships at work (1-4) (Ordinal).
- **StandardHours:** Standard working hours (usually 80, a fixed value) (Numerical, constant).
- **StockOptionLevel:** Level of stock options granted (0-3) (Ordinal).
- **TotalWorkingYears:** Total years of work experience (Numerical).
- **TrainingTimesLastYear:** Number of training sessions attended in the last year (Numerical).
- **WorkLifeBalance:** Work-life balance rating (1-4) (Ordinal).
- **YearsAtCompany:** Number of years spent at the current company (Numerical).
- **YearsInCurrentRole:** Number of years in the current role (Numerical).
- **YearsSinceLastPromotion:** Years since the last promotion (Numerical).
- **YearsWithCurrManager:** Number of years working with the current manager (Numerical).

Data Preprocessing

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import LabelEncoder
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import confusion_matrix, classification_report
import seaborn as sns

df = pd.read_csv(r"C:\Users\aditi\Downloads\DAV_dataset\WA_Fn-UseC_-HR-Employee-Attrition.csv")

df.head(10)
```

This code sets up a machine learning environment for analysing an employee attrition dataset. It imports libraries like pandas and numpy for data handling, scikit-learn for data preprocessing, and classifiers like RandomForest, DecisionTree, and LogisticRegression. The dataset is loaded from a CSV file and displayed to preview the first 10 rows.

	Age	Attrition	BusinessTravel	DailyRate	Department	DistanceFromHome	Education	EducationField	EmployeeCount	EmployeeNumber	...	Relations
0	41	Yes	Travel_Rarely	1102	Sales	1	2	Life Sciences	1	1	...	
1	49	No	Travel_Frequently	279	Research & Development	8	1	Life Sciences	1	2	...	
2	37	Yes	Travel_Rarely	1373	Research & Development	2	2	Other	1	4	...	
3	33	No	Travel_Frequently	1392	Research & Development	3	4	Life Sciences	1	5	...	
4	27	No	Travel_Rarely	591	Research & Development	2	1	Medical	1	7	...	
5	32	No	Travel_Frequently	1005	Research & Development	2	2	Life Sciences	1	8	...	
6	59	No	Travel_Rarely	1324	Research & Development	3	3	Medical	1	10	...	
7	30	No	Travel_Rarely	1358	Research & Development	24	1	Life Sciences	1	11	...	
8	38	No	Travel_Frequently	216	Research & Development	23	3	Life Sciences	1	12	...	
9	36	No	Travel_Rarely	1299	Research & Development	27	3	Medical	1	13	...	

10 rows × 35 columns

```
df.info()
```

The `df.info()` command provides a summary of the DataFrame, showing the data types, non-null counts, and memory usage for each column.

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1470 entries, 0 to 1469
Data columns (total 35 columns):
#   Column              Non-Null Count  Dtype
---  -
0   Age                  1470 non-null  int64
1   Attrition            1470 non-null  object
2   BusinessTravel       1470 non-null  object
3   DailyRate            1470 non-null  int64
4   Department           1470 non-null  object
5   DistanceFromHome     1470 non-null  int64
6   Education             1470 non-null  int64
7   EducationField        1470 non-null  object
8   EmployeeCount        1470 non-null  int64
9   EmployeeNumber       1470 non-null  int64
10  EnvironmentSatisfaction 1470 non-null  int64
11  Gender               1470 non-null  object
12  HourlyRate           1470 non-null  int64
13  JobInvolvement       1470 non-null  int64
14  JobLevel             1470 non-null  int64
15  JobRole              1470 non-null  object
16  JobSatisfaction      1470 non-null  int64
17  MaritalStatus        1470 non-null  object
18  MonthlyIncome        1470 non-null  int64
19  MonthlyRate          1470 non-null  int64
20  NumCompaniesWorked   1470 non-null  int64
21  Over18               1470 non-null  object
22  OverTime             1470 non-null  object
23  PercentSalaryHike    1470 non-null  int64
24  PerformanceRating    1470 non-null  int64
25  RelationshipSatisfaction 1470 non-null  int64
26  StandardHours        1470 non-null  int64
27  StockOptionLevel     1470 non-null  int64
28  TotalWorkingYears    1470 non-null  int64
29  TrainingTimesLastYear 1470 non-null  int64
30  WorkLifeBalance      1470 non-null  int64
31  YearsAtCompany       1470 non-null  int64
32  YearsInCurrentRole   1470 non-null  int64
33  YearsSinceLastPromotion 1470 non-null  int64
34  YearsWithCurrManager 1470 non-null  int64
dtypes: int64(26), object(9)
memory usage: 402.1+ KB

```

```

print(df.isnull().sum())
Age                  0
Attrition            0
BusinessTravel       0
DailyRate            0
Department           0
DistanceFromHome     0
Education             0
EducationField        0
EmployeeCount        0
EmployeeNumber       0
EnvironmentSatisfaction 0
Gender               0
HourlyRate           0
JobInvolvement       0
JobLevel             0
JobRole              0
JobSatisfaction      0
MaritalStatus        0
MonthlyIncome        0
MonthlyRate          0
NumCompaniesWorked   0
Over18               0
OverTime             0
PercentSalaryHike    0
PerformanceRating    0
RelationshipSatisfaction 0
StandardHours        0
StockOptionLevel     0
TotalWorkingYears    0
TrainingTimesLastYear 0
WorkLifeBalance      0
YearsAtCompany       0
YearsInCurrentRole   0
YearsSinceLastPromotion 0
YearsWithCurrManager 0
dtype: int64

```

The `print(df.isnull().sum())` command shows the number of missing (null) values in each column of the DataFrame.

```
df.describe()
```

	Age	DailyRate	DistanceFromHome	Education	EmployeeCount	EmployeeNumber	EnvironmentSatisfaction	HourlyRate	JobInvolvement
count	1470.000000	1470.000000	1470.000000	1470.000000	1470.0	1470.000000	1470.000000	1470.000000	1470.000000
mean	36.923810	802.485714	9.192517	2.912925	1.0	1024.865306	2.721769	65.891156	2.729930
std	9.135373	403.509100	8.106864	1.024165	0.0	602.024335	1.093082	20.329428	0.711156
min	18.000000	102.000000	1.000000	1.000000	1.0	1.000000	1.000000	30.000000	1.000000
25%	30.000000	465.000000	2.000000	2.000000	1.0	491.250000	2.000000	48.000000	2.000000
50%	36.000000	802.000000	7.000000	3.000000	1.0	1020.500000	3.000000	66.000000	3.000000
75%	43.000000	1157.000000	14.000000	4.000000	1.0	1555.750000	4.000000	83.750000	3.000000
max	60.000000	1499.000000	29.000000	5.000000	1.0	2068.000000	4.000000	100.000000	4.000000

8 rows x 26 columns

The `df.describe()` command provides summary statistics (like mean, median, standard deviation, min, and max) for all numerical columns in the DataFrame.

```

if df['Over18'].nunique() == 1:
    df.drop('Over18', axis=1, inplace=True)
    print("'Over18' column dropped as all values are the same.")
else:
    print("'Over18' column contains different values.")

```

'Over18' column dropped as all values are the same.

This code checks if the column “Over18” in the DataFrame has only one unique value. If so, it drops the column because it's redundant (contains the same value for all rows). Otherwise, it indicates that the column has varying values.

Outlier Detection

```

##Outlier Detection

def detect_outliers_iqr(df):
    outlier_counts = {}
    for col in df.select_dtypes(include=[np.number]):
        Q1 = df[col].quantile(0.25)
        Q3 = df[col].quantile(0.75)
        IQR = Q3 - Q1
        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR
        outliers = df[(df[col] < lower_bound) | (df[col] > upper_bound)]
        outlier_counts[col] = len(outliers)
    return outlier_counts
outliers_iqr = detect_outliers_iqr(df)

print("\n Total Outliers detected using IQR method:")
for col, count in outliers_iqr.items():
    print(f"{col}: {count} outliers")

```

This code detects outliers in numerical columns using the IQR method, identifying values below ‘ $Q1 - 1.5 * IQR$ ’ or above ‘ $Q3 + 1.5 * IQR$ ’. It counts outliers for each column and prints the results.

```

    Total Outliers detected using IQR method:
Age: 0 outliers
DailyRate: 0 outliers
DistanceFromHome: 0 outliers
Education: 0 outliers
EmployeeCount: 0 outliers
EmployeeNumber: 0 outliers
EnvironmentSatisfaction: 0 outliers
HourlyRate: 0 outliers
JobInvolvement: 0 outliers
JobLevel: 0 outliers
JobSatisfaction: 0 outliers
MonthlyIncome: 114 outliers
MonthlyRate: 0 outliers
NumCompaniesWorked: 52 outliers
PercentSalaryHike: 0 outliers
PerformanceRating: 226 outliers
RelationshipSatisfaction: 0 outliers
StandardHours: 0 outliers
StockOptionLevel: 85 outliers
TotalWorkingYears: 63 outliers
TrainingTimesLastYear: 238 outliers
WorkLifeBalance: 0 outliers
YearsAtCompany: 104 outliers
YearsInCurrentRole: 21 outliers
YearsSinceLastPromotion: 107 outliers
YearsWithCurrManager: 14 outliers

```

Box Plots

```

numeric_columns = df.select_dtypes(include=[np.number]).columns
num_cols = len(numeric_columns)

num_rows = (num_cols + 2) // 3
fig, axes = plt.subplots(num_rows, 3, figsize=(15, 5 * num_rows))
fig.suptitle("Box Plots of Numeric Columns", fontsize=16)

axes = axes.flatten()

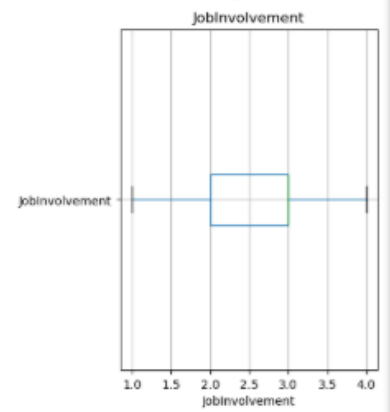
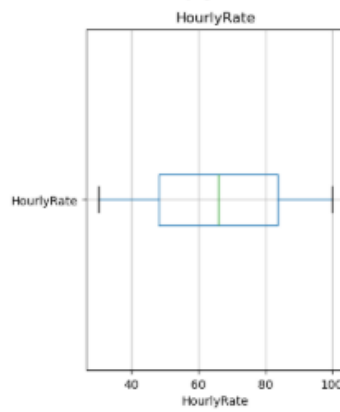
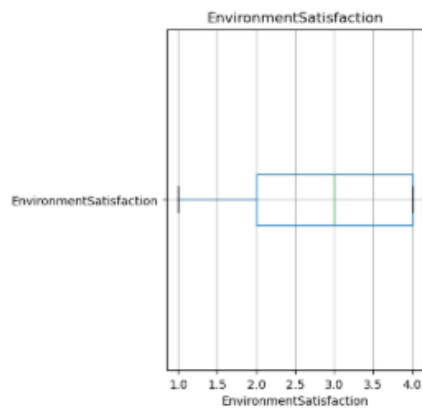
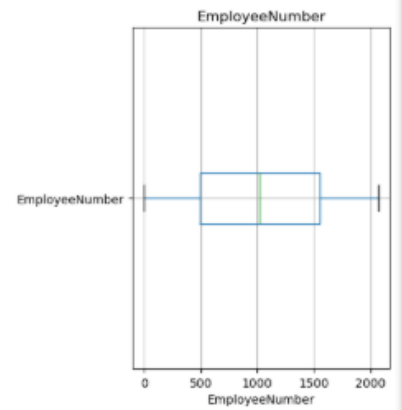
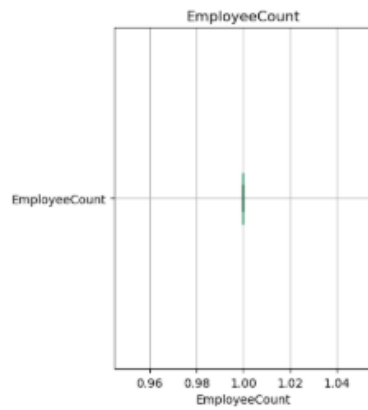
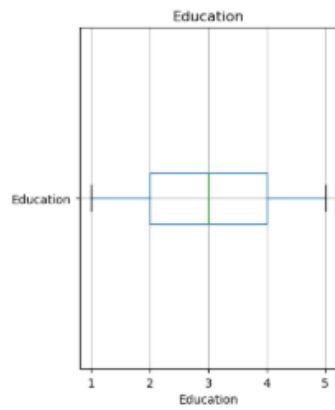
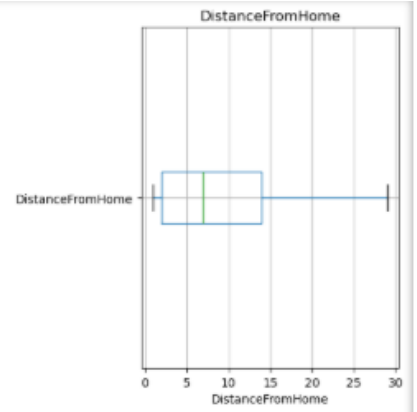
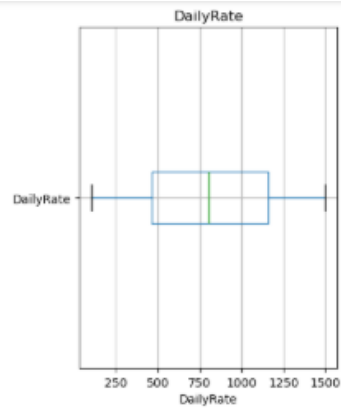
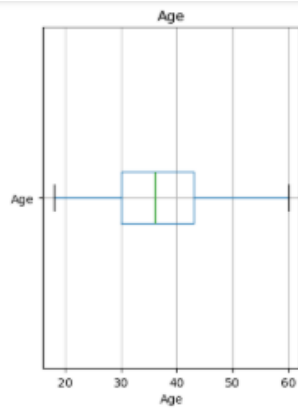
for ax, col in zip(axes, numeric_columns):
    df.boxplot(column=col, ax=ax, vert=False)
    ax.set_title(f'{col}')
    ax.set_xlabel(col)

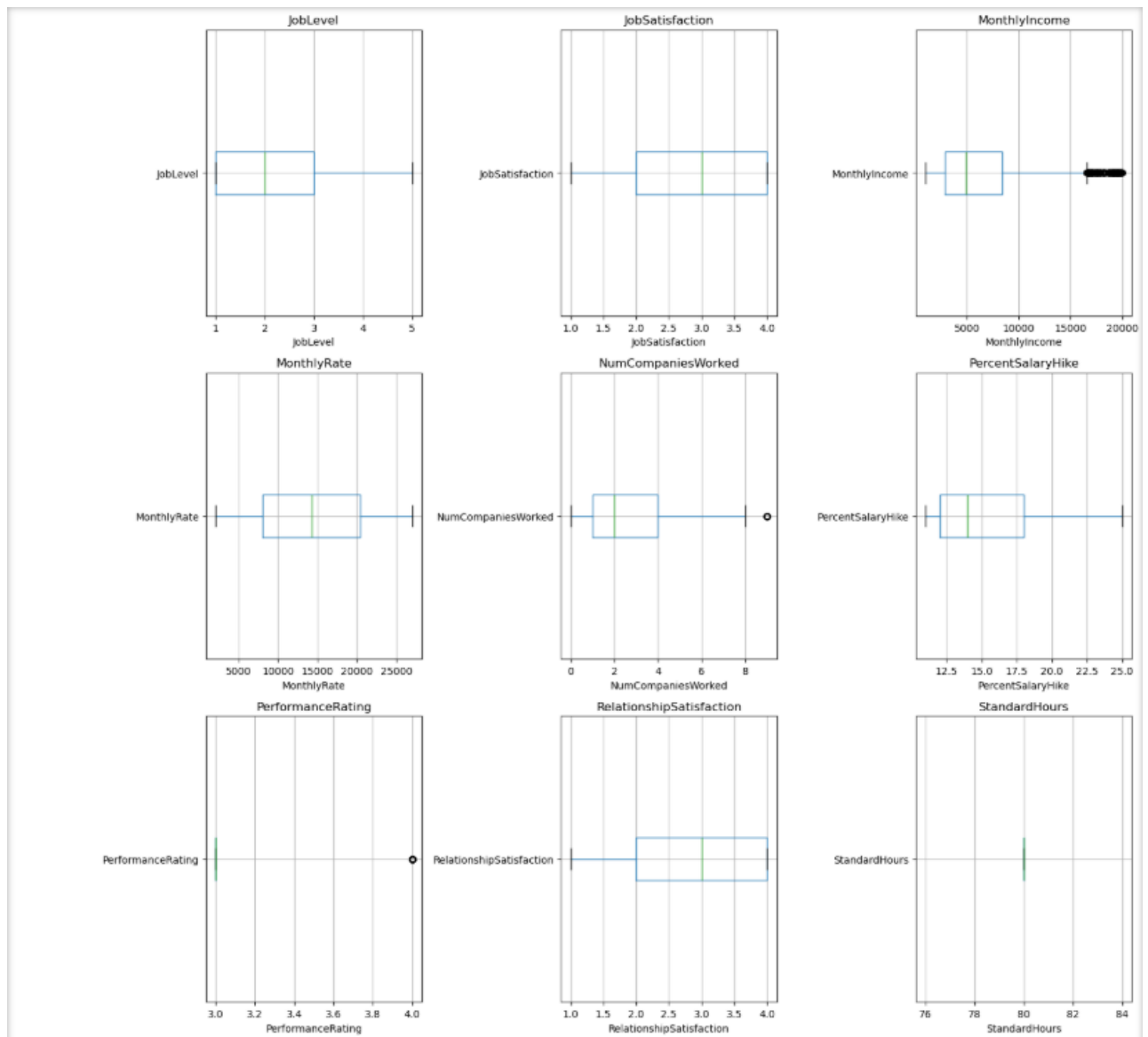
for i in range(len(numeric_columns), len(axes)):
    fig.delaxes(axes[i])

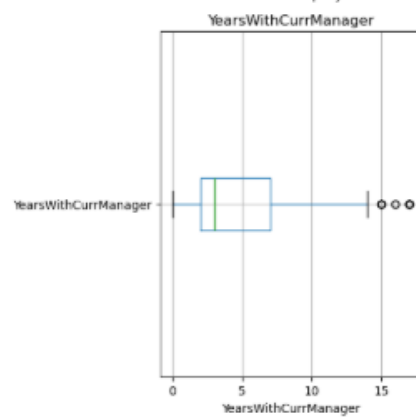
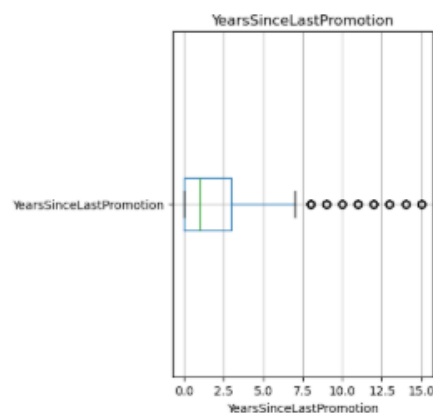
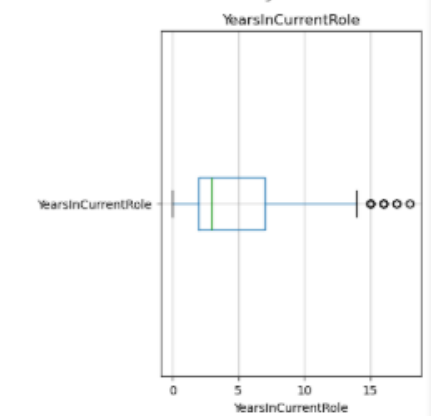
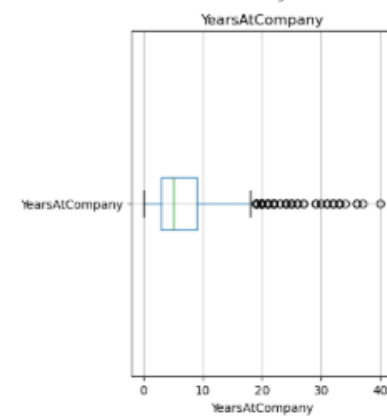
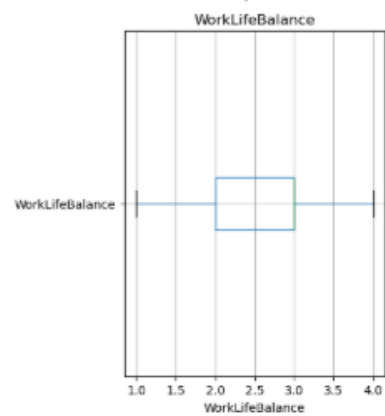
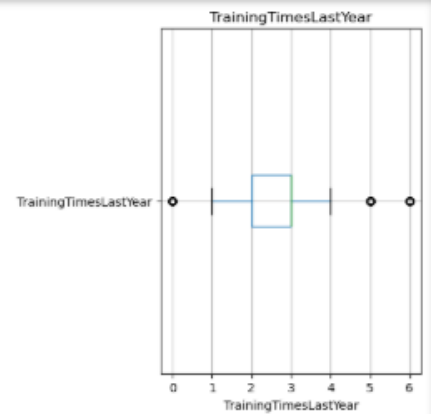
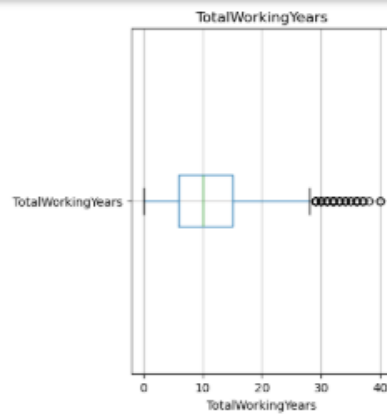
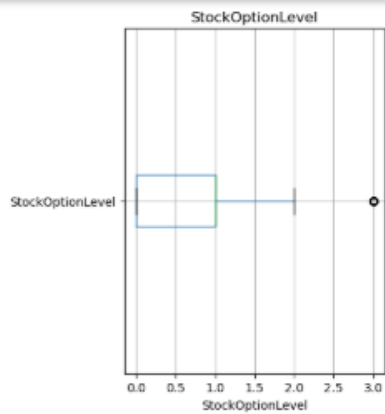
plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()

```

This code generates box plots for all numeric columns in a DataFrame (df). It first identifies the numeric columns and calculates the required number of rows for the plots. Then, it creates subplots, where each subplot corresponds to a numeric column's box plot. The code iterates over the numeric columns, plots them, and sets titles and labels.







```
def remove_outliers_iqr(df):
    outlier_indices = set()
    for col in df.select_dtypes(include=[np.number]):
        Q1 = df[col].quantile(0.25)
        Q3 = df[col].quantile(0.75)
        IQR = Q3 - Q1
        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR

        outliers = df[(df[col] < lower_bound) | (df[col] > upper_bound)].index
        outlier_indices.update(outliers)

    return df.drop(index=outlier_indices), len(outlier_indices)

df_cleaned, total_removed = remove_outliers_iqr(df)

print(f"\nTotal Outliers removed: {total_removed}")
df_cleaned.info()
```

This code defines a function 'remove_outliers_iqr' that removes outliers from a DataFrame ('df') using the Interquartile Range (IQR) method. For each numeric column, it calculates the first (Q1) and third quartiles (Q3), computes the IQR, and determines the lower and upper bounds for outliers. Rows with values outside these bounds are considered outliers and are removed from the DataFrame. The function returns the cleaned DataFrame and the total number of removed outliers. The cleaned DataFrame's info is printed along with the total number of outliers removed.

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 1470 entries, 0 to 1469
```

```
Data columns (total 35 columns):
```

#	Column	Non-Null Count	Dtype
0	Age	1470 non-null	int64
1	Attrition	1470 non-null	object
2	BusinessTravel	1470 non-null	object
3	DailyRate	1470 non-null	int64
4	Department	1470 non-null	object
5	DistanceFromHome	1470 non-null	int64
6	Education	1470 non-null	int64
7	EducationField	1470 non-null	object
8	EmployeeCount	1470 non-null	int64
9	EmployeeNumber	1470 non-null	int64
10	EnvironmentSatisfaction	1470 non-null	int64
11	Gender	1470 non-null	object
12	HourlyRate	1470 non-null	int64
13	JobInvolvement	1470 non-null	int64
14	JobLevel	1470 non-null	int64
15	JobRole	1470 non-null	object
16	JobSatisfaction	1470 non-null	int64
17	MaritalStatus	1470 non-null	object
18	MonthlyIncome	1470 non-null	int64
19	MonthlyRate	1470 non-null	int64
20	NumCompaniesWorked	1470 non-null	int64
21	Over18	1470 non-null	object
22	OverTime	1470 non-null	object
23	PercentSalaryHike	1470 non-null	int64
24	PerformanceRating	1470 non-null	int64
25	RelationshipSatisfaction	1470 non-null	int64
26	StandardHours	1470 non-null	int64
27	StockOptionLevel	1470 non-null	int64
28	TotalWorkingYears	1470 non-null	int64
29	TrainingTimesLastYear	1470 non-null	int64
30	WorkLifeBalance	1470 non-null	int64
31	YearsAtCompany	1470 non-null	int64
32	YearsInCurrentRole	1470 non-null	int64
33	YearsSinceLastPromotion	1470 non-null	int64
34	YearsWithCurrManager	1470 non-null	int64

```
dtypes: int64(26), object(9)
```

```
memory usage: 402.1+ KB
```

Label Encoding of Binary Columns in a DataFrame

```
#Label Encoding of binary columns

binary_columns = [col for col in df_cleaned.columns if df_cleaned[col].nunique() == 2]

print("Binary columns in the dataset:")
print(binary_columns)
label_encoder = LabelEncoder()

binary_mappings = {}

for col in binary_columns:
    df_cleaned[col] = label_encoder.fit_transform(df_cleaned[col])

    mapping = dict(zip(label_encoder.classes_, label_encoder.transform(label_encoder.classes_)))
    binary_mappings[col] = mapping

print("\nMappings for binary columns:")
for col, mapping in binary_mappings.items():
    print(f"{col}: {mapping}")
df_cleaned.head(10)
```

- Identifies binary columns by checking the number of unique values.
- Applies label encoding to binary columns.
- Saves and prints the mappings for each binary column.
- Displays the first 10 rows of the cleaned DataFrame after encoding.

```
Binary columns in the dataset:
['Attrition', 'Gender', 'OverTime']
```

```
Mappings for binary columns:
Attrition: {'No': 0, 'Yes': 1}
Gender: {'Female': 0, 'Male': 1}
OverTime: {'No': 0, 'Yes': 1}
```

BusinessTravel	DailyRate	Department	DistanceFromHome	Education	EducationField	EmployeeCount	EmployeeNumber	...	RelationshipSatisfaction
Travel_Rarely	1373	Research & Development	2	2	Other	1	4	...	2
Travel_Frequently	1392	Research & Development	3	4	Life Sciences	1	5	...	3
Travel_Frequently	1005	Research & Development	2	2	Life Sciences	1	8	...	3
Travel_Rarely	1299	Research & Development	27	3	Medical	1	13	...	2
Travel_Rarely	153	Research & Development	15	2	Life Sciences	1	15	...	4
Travel_Rarely	670	Research & Development	26	1	Life Sciences	1	16	...	4
Travel_Rarely	1346	Research & Development	19	2	Medical	1	18	...	3
Travel_Rarely	103	Research & Development	24	3	Life Sciences	1	19	...	2
Non-Travel	1123	Research & Development	16	2	Medical	1	22	...	2
Travel_Rarely	371	Research & Development	2	3	Life Sciences	1	24	...	3

Label Encoding of Multi-Class Categorical Columns

```
#Label Encoding of multi-class categorical columns

multi_class_columns = [col for col in df_cleaned.columns if df_cleaned[col].nunique() > 2 and
df_cleaned[col].dtype == 'object']

label_encoder = LabelEncoder()

multi_class_mappings = {}

for col in multi_class_columns:
    df_cleaned[col] = label_encoder.fit_transform(df_cleaned[col])

    mapping = dict(zip(label_encoder.classes_, label_encoder.transform(label_encoder.classes_)))
    multi_class_mappings[col] = mapping

print("\nMappings for multi-class columns:")
for col, mapping in multi_class_mappings.items():
    print(f"{col}: {mapping}")
df_cleaned.head(10)
```

- Identifies multi-class categorical columns (more than two unique values and of type 'object').
- Applies label encoding to convert categorical values into numerical labels
- Stores and prints the mappings of original values to encoded labels for each multi-class column.

Mappings for multi-class columns:

BusinessTravel: {'Non-Travel': 0, 'Travel_Frequently': 1, 'Travel_Rarely': 2}

Department: {'Human Resources': 0, 'Research & Development': 1, 'Sales': 2}

EducationField: {'Human Resources': 0, 'Life Sciences': 1, 'Marketing': 2, 'Medical': 3, 'Other': 4, 'Technical Degree': 5}

JobRole: {'Healthcare Representative': 0, 'Human Resources': 1, 'Laboratory Technician': 2, 'Manager': 3, 'Manufacturing Director': 4, 'Research Director': 5, 'Research Scientist': 6, 'Sales Executive': 7, 'Sales Representative': 8}

MaritalStatus: {'Divorced': 0, 'Married': 1, 'Single': 2}

Mappings for each multi-class column showing original categories and their encoded labels.

First 10 rows of the cleaned DataFrame after encoding are shown below

	Age	Attrition	BusinessTravel	DailyRate	Department	DistanceFromHome	Education	EducationField	EmployeeCount	EmployeeNumber	...	Relati
2	37	1	2	1373	1	2	2	4	1	4	...	
3	33	0	1	1392	1	3	4	1	1	5	...	
5	32	0	1	1005	1	2	2	1	1	8	...	
9	36	0	2	1299	1	27	3	3	1	13	...	
11	29	0	2	153	1	15	2	1	1	15	...	
12	31	0	2	670	1	26	1	1	1	16	...	
13	34	0	2	1346	1	19	2	3	1	18	...	
14	28	1	2	103	1	24	3	1	1	19	...	
17	22	0	0	1123	1	16	2	3	1	22	...	
19	38	0	2	371	1	2	3	1	1	24	...	

10 rows × 34 columns

Min-Max Scaling of Numeric Columns

```
#Min-Max Scaling of numerical columns
df_min_max = df_cleaned.copy()
numeric_columns = df_cleaned.select_dtypes(include=[np.number]).columns
min_max_scaler = MinMaxScaler()
df_min_max[numeric_columns] = min_max_scaler.fit_transform(df_min_max[numeric_columns])
df_min_max.head(10)
df_min_max.to_csv('df_min_max.csv', index=False)
import IPython
IPython.display.FileLink('df_min_max.csv')

df = pd.read_csv(r"C:\Users\aditi\Downloads\DAV_dataset\WA_Fn-UseC_-HR-Employee-Attrition.csv")
df.info()
```

This code applies Min-Max scaling to the numeric columns of the cleaned DataFrame (df_cleaned). It first creates a copy of the original DataFrame (df_min_max) and identifies the numeric columns. The MinMaxScaler from sklearn.preprocessing is then used to scale each numeric column to the range [0, 1], transforming the values and updating them in the new DataFrame. This ensures that all numeric values are on the same scale.

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1470 entries, 0 to 1469
Data columns (total 35 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Age                                   1470 non-null   int64
1   Attrition                           1470 non-null   object
2   BusinessTravel                       1470 non-null   object
3   DailyRate                           1470 non-null   int64
4   Department                           1470 non-null   object
5   DistanceFromHome                    1470 non-null   int64
6   Education                           1470 non-null   int64
7   EducationField                       1470 non-null   object
8   EmployeeCount                       1470 non-null   int64
9   EmployeeNumber                      1470 non-null   int64
10  EnvironmentSatisfaction              1470 non-null   int64
11  Gender                              1470 non-null   object
12  HourlyRate                          1470 non-null   int64
13  JobInvolvement                      1470 non-null   int64
14  JobLevel                            1470 non-null   int64
15  JobRole                             1470 non-null   object
16  JobSatisfaction                     1470 non-null   int64
17  MaritalStatus                       1470 non-null   object
18  MonthlyIncome                       1470 non-null   int64
19  MonthlyRate                         1470 non-null   int64
20  NumCompaniesWorked                  1470 non-null   int64
21  Over18                              1470 non-null   object
22  OverTime                            1470 non-null   object
23  PercentSalaryHike                   1470 non-null   int64
24  PerformanceRating                   1470 non-null   int64
25  RelationshipSatisfaction             1470 non-null   int64
26  StandardHours                       1470 non-null   int64
27  StockOptionLevel                    1470 non-null   int64
28  TotalWorkingYears                   1470 non-null   int64
29  TrainingTimesLastYear               1470 non-null   int64
30  WorkLifeBalance                     1470 non-null   int64
31  YearsAtCompany                      1470 non-null   int64
32  YearsInCurrentRole                  1470 non-null   int64
33  YearsSinceLastPromotion              1470 non-null   int64
34  YearsWithCurrManager                 1470 non-null   int64
dtypes: int64(26), object(9)
memory usage: 402.1+ KB

```

Visualization using Tableau

Employee Count	Attrition Count	Attrition Rate	Active Employees	Avg. Age
1,470	237	16.12%	1,233	37

1. Employee Count – 1,470

- Represents the total number of employees in the organization, including both active and exited staff.
- Acts as the base metric for calculating other KPIs like attrition rate and active employees.

2. Attrition Count – 237

- Total number of employees who have left the organization during the analysis period.
- A count of 237 suggests a noticeable employee turnover, impacting overall productivity.

3. Attrition Rate – 16.12%

- Shows the percentage of employees who left compared to the total workforce.
- A rate of 16.12% is slightly higher than ideal, signaling potential retention or satisfaction issues.

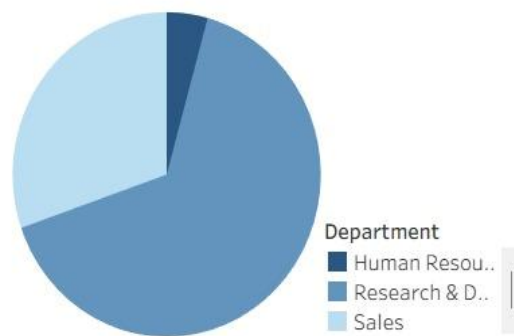
4. Active Employees – 1,233

- Represents the current number of employees remaining after attrition.
- Indicates that the company has retained about 84% of its total workforce.

5. Average Age – 37 years

- The average age of employees is 37, suggesting a mid-career workforce.
- Such employees value career growth, stability, and work-life balance.

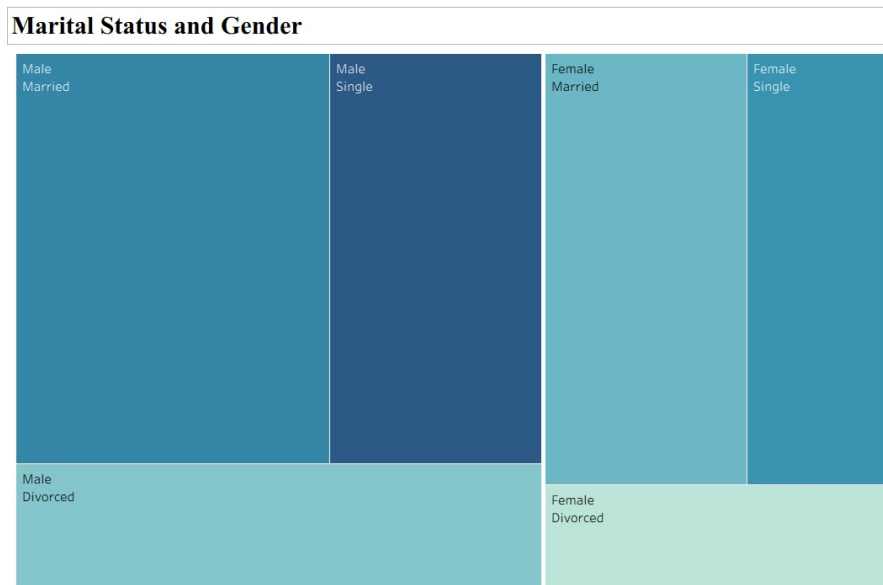
Department wise Attrition



This pie chart breaks down the attrition (employee exits) across different departments:

- Research & Development (R&D) holds the largest portion of attrition.
- Sales follows next.
- Human Resources (HR) has the lowest attrition.

Management should prioritize retention strategies in Sales and R&D — such as better incentive structures, learning opportunities, and work-life balance programs.



The treemap visualizes attrition distribution across gender (male/female) and marital status (married/single) categories.

- Married males form the largest block — showing they have the highest attrition rate.
- Single males are next, followed by married females and single females.
- Overall, males contribute more to attrition than females.

Gender and marital status patterns reveal where targeted retention measures may be most effective — such as flexible schedules, family benefits, and career growth incentives.

Department and Job role by Gender

Department	Job Role	Gender	
		Female	Male
Human Resources	Human Resources	16	36
	Manager	4	7
Research & Development	Healthcare Represe..	51	80
	Laboratory Technici..	85	174
	Manager	24	30
	Manufacturing Direc..	72	73
	Research Director	33	47
	Research Scientist	114	178
Sales	Manager	19	18
	Sales Executive	132	194
	Sales Representative	38	45

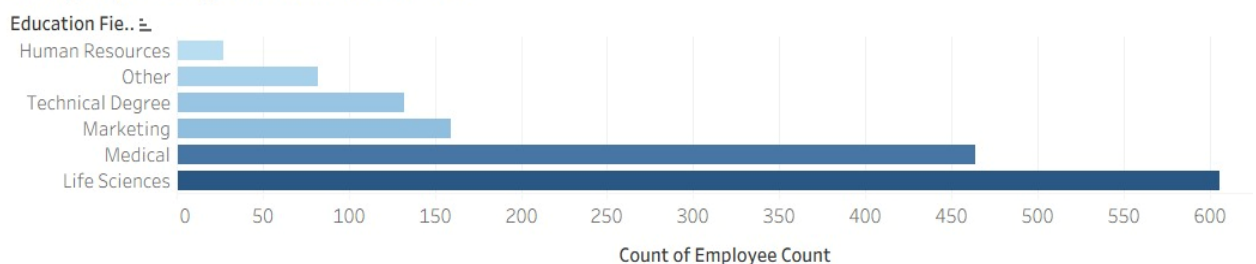
This table breaks down:

- Departments
- Job roles within departments
- Gender distribution (Female vs Male)

Detailed Observations:

- R&D has the highest overall employee count (both male and female).
- Sales also has a large workforce, with males dominating.
- Female representation:
 - Strong in lab/research roles
 - Weak in managerial and manufacturing roles
- Male employees dominate technical, sales, and managerial positions.

Employees by Education Field



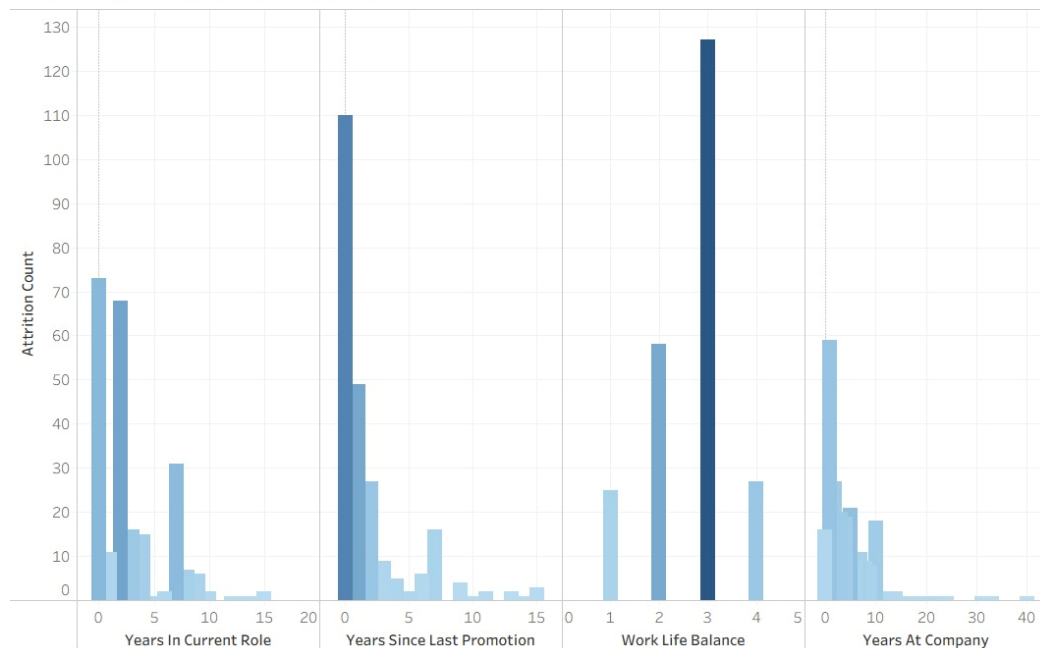
This bar chart represents employee count across educational backgrounds:

- Life Sciences
- Medical
- Marketing
- Technical Degree
- Human Resources
- Others

Detailed Observations:

- Most employees come from Life Sciences and Medical fields, suggesting the company likely operates in healthcare, pharmaceuticals, or biotechnology.
- Fields like HR and Marketing have fewer employees, reflecting smaller support departments.

Various parameters to Attrition



a. Years in Current Role vs Attrition

- The histogram shows the count of employees who left based on how long they've been in their current role.
- Attrition is highest between 0–5 years, particularly in the first 2 years. Employees tend to leave early — possibly due to misaligned expectations, poor onboarding, or lack of growth.

b. Years Since Last Promotion vs Attrition

- Most attrition occurs when employees have had 1–3 years since their last promotion.
- Lack of promotion or perceived stagnation often triggers exits.

c. Work-Life Balance vs Attrition

- Attrition peaks at Work-Life Balance rating = 3 (on a scale likely from 1 to 4). Employees who rate work-life balance as “average” may still feel overburdened or unfulfilled.

d. Years at Company vs Attrition

- Attrition is concentrated among those with less than 10 years of service. Early-career employees are leaving more often, while long-tenured employees tend to stay.

Final Dashboard

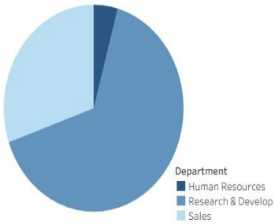
HR ANALYTICS DASHBOARD

Employee Count	Attrition Count	Attrition Rate	Active Employees	Avg. Age
1,470	237	16.12%	1,233	37

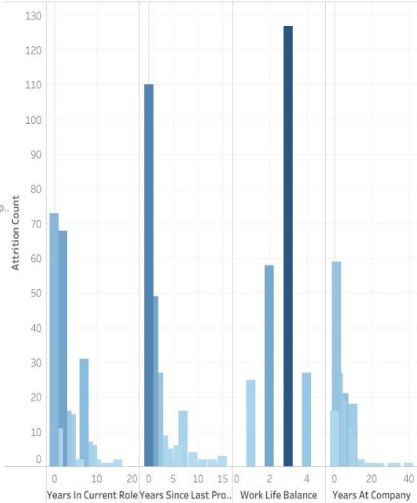
Department and Job role by Gender

Department	Job Role	Female	Male
Human Resources	Human Resources Manager	16	36
Research & Development	Healthcare Representative	4	7
	Laboratory Technician	51	80
	Manager	85	174
	Manufacturing Director	24	30
	Research Director	72	73
	Research Scientist	33	47
Sales	Manager	114	178
	Sales Executive	19	18
	Sales Representative	132	194
		38	45

Department wise Attrition



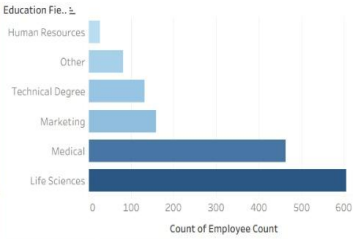
Various parameters to Attrition



Marital Status and Gender



Employees by Education Field



Applying Machine Learning Model

- **Filtering Columns Based on Correlation with Target Variable**

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import classification_report, accuracy_score, confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns

np.random.seed(42)
df = pd.DataFrame({
    'Attrition': np.random.randint(0, 2, 200),
    'Age': np.random.randint(21, 60, 200),
    'MonthlyIncome': np.random.randint(2500, 20000, 200),
    'JobSatisfaction': np.random.randint(1, 5, 200),
    'DistanceFromHome': np.random.randint(1, 30, 200),
})

#Numeric columns for correlation
numeric_df = df.select_dtypes(include=[np.number])

# Computing correlation matrix and select columns with abs(correlation) > 0.1 with target
correlation_matrix = numeric_df.corr()
correlation_with_target = correlation_matrix['Attrition'].abs()
selected_columns = correlation_with_target[correlation_with_target > 0.1].index
print(f'Selected columns with abs(correlation) > 0.1: {selected_columns.tolist()}')

df_filtered = df[selected_columns]
```

- **Splitting Data into Training and Testing Sets**

```
# Split data
X = df_filtered.drop('Attrition', axis=1)
y = df_filtered['Attrition']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

- **Model Evaluation and Comparison of Classifiers**

```
# Models to compare
models = {
    "Logistic Regression": LogisticRegression(),
    "Random Forest": RandomForestClassifier(),
    "SVM": SVC(),
    "Decision Tree": DecisionTreeClassifier(),
    "K-Nearest Neighbors": KNeighborsClassifier()
}

metrics = {
    "Model": [],
    "Accuracy": [],
    "Precision": [],
    "Recall": [],
    "F1-Score": []
}

def evaluate_model(model, X_train, X_test, y_train, y_test):
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    report = classification_report(y_test, y_pred, output_dict=True)
    print(f'\nClassification Report for {model.__class__.__name__}:')
    print(classification_report(y_test, y_pred))

    metrics["Model"].append(model.__class__.__name__)
    metrics["Accuracy"].append(accuracy_score(y_test, y_pred))
    metrics["Precision"].append(report.get("weighted avg", {}).get("precision", 0))
    metrics["Recall"].append(report.get("weighted avg", {}).get("recall", 0))
    metrics["F1-Score"].append(report.get("weighted avg", {}).get("f1-score", 0))
```

```

cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt="d", cmap="YlGnBu")
plt.title(f'Confusion Matrix for {model.__class__.__name__}')
plt.ylabel('Actual')
plt.xlabel('Predicted')
plt.show()

for model_name, model in models.items():
    print(f"\nEvaluating {model_name}...")
    evaluate_model(model, X_train, X_test, y_train, y_test)

metrics_df = pd.DataFrame(metrics)

if not metrics_df.empty:
    metrics_df.set_index("Model").plot(kind="bar", figsize=(12, 8), colormap='YlGnBu')
    plt.title('Comparison of Classifier Metrics (Accuracy, Precision, Recall, F1-Score)', fontsize=16)
    plt.ylabel('Score')
    plt.xticks(rotation=45, ha="right")
    plt.grid(True, axis='y', linestyle='--', alpha=0.7)
    plt.show()
else:
    print("No metrics to display.")

```

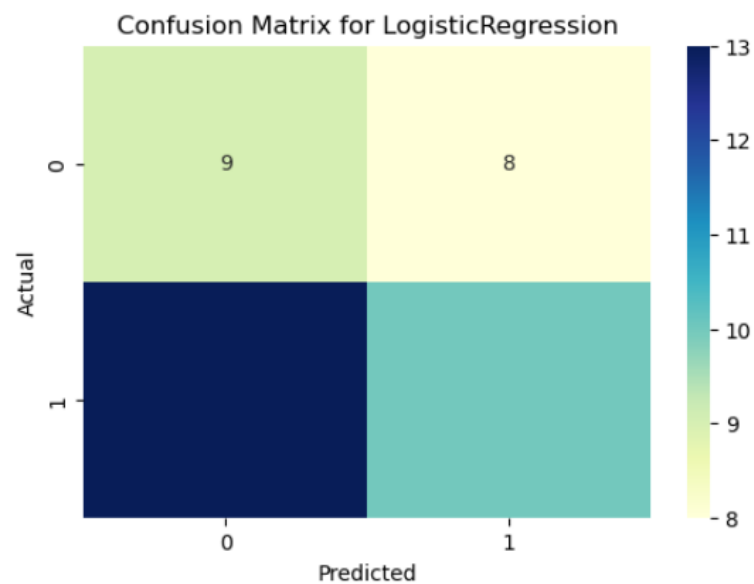
- This code evaluates the performance of multiple machine learning models (Logistic Regression, Random Forest, SVM, Decision Tree, and K-Nearest Neighbors) on a classification task.
- It trains each model using a given dataset, makes predictions, and generates a classification report to assess the accuracy, precision, recall, and F1-score.
- It also visualises a confusion matrix for each model to show how well the model predicted each class.
- Finally, the code compares the models' performance by plotting a bar chart of the evaluation metrics.
- This helps in understanding which model performs best for the given dataset.

Selected columns with $\text{abs}(\text{correlation}) > 0.1$: ['Attrition', 'DistanceFromHome']

Evaluating Logistic Regression...

Classification Report for LogisticRegression:

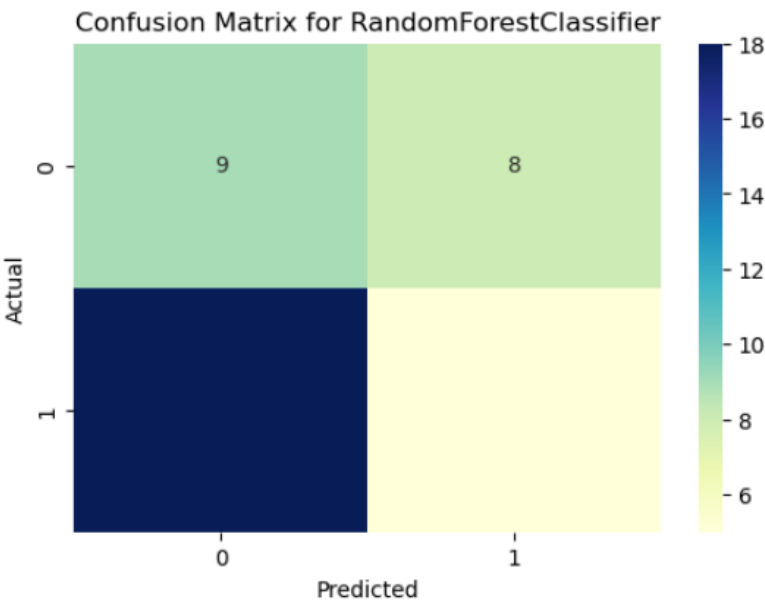
	precision	recall	f1-score	support
0	0.41	0.53	0.46	17
1	0.56	0.43	0.49	23
accuracy			0.48	40
macro avg	0.48	0.48	0.47	40
weighted avg	0.49	0.47	0.48	40



Evaluating Random Forest...

Classification Report for RandomForestClassifier:

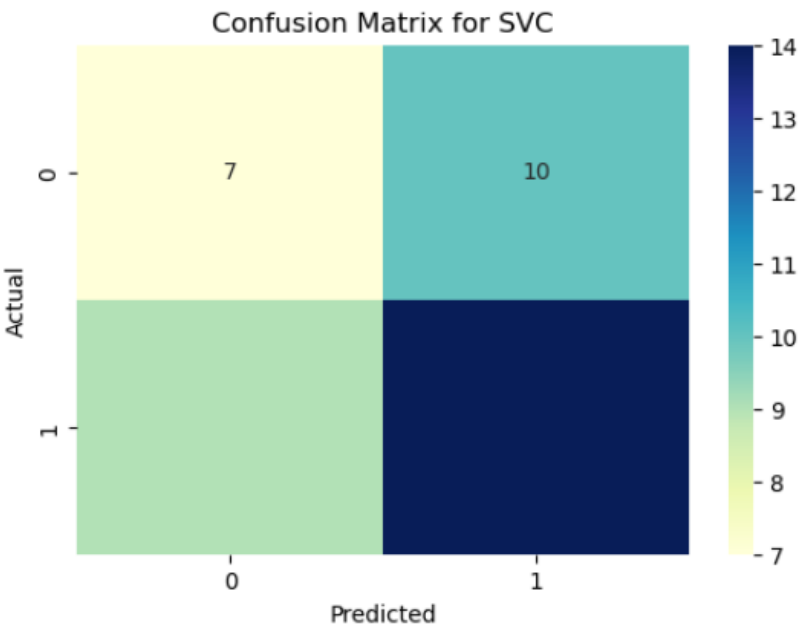
	precision	recall	f1-score	support
0	0.33	0.53	0.41	17
1	0.38	0.22	0.28	23
accuracy			0.35	40
macro avg	0.36	0.37	0.34	40
weighted avg	0.36	0.35	0.33	40



Evaluating SVM...

Classification Report for SVC:

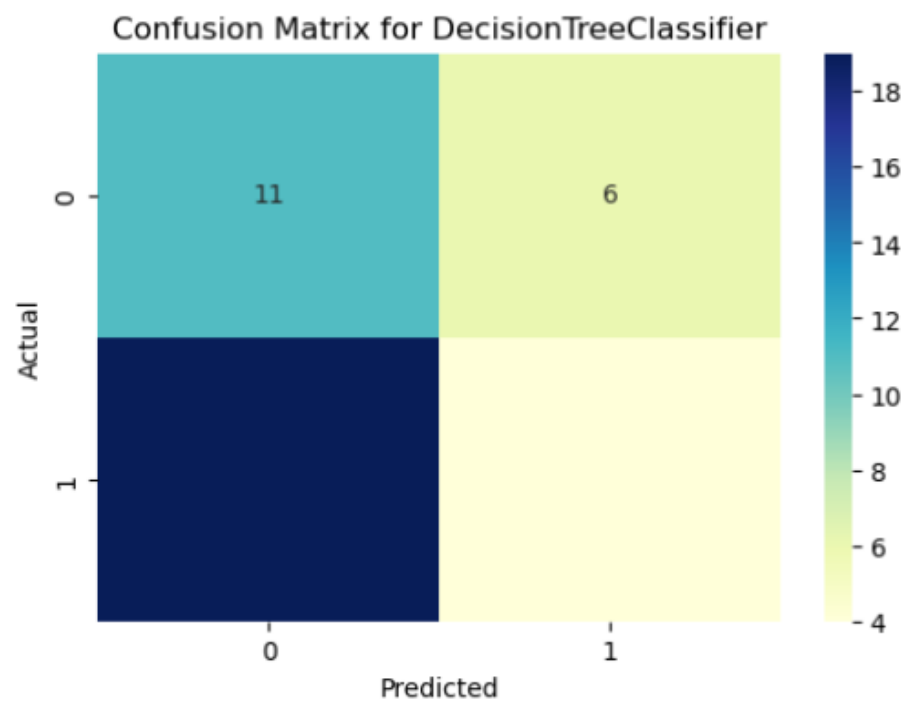
	precision	recall	f1-score	support
0	0.44	0.41	0.42	17
1	0.58	0.61	0.60	23
accuracy			0.53	40
macro avg	0.51	0.51	0.51	40
weighted avg	0.52	0.53	0.52	40



Evaluating Decision Tree...

Classification Report for DecisionTreeClassifier:

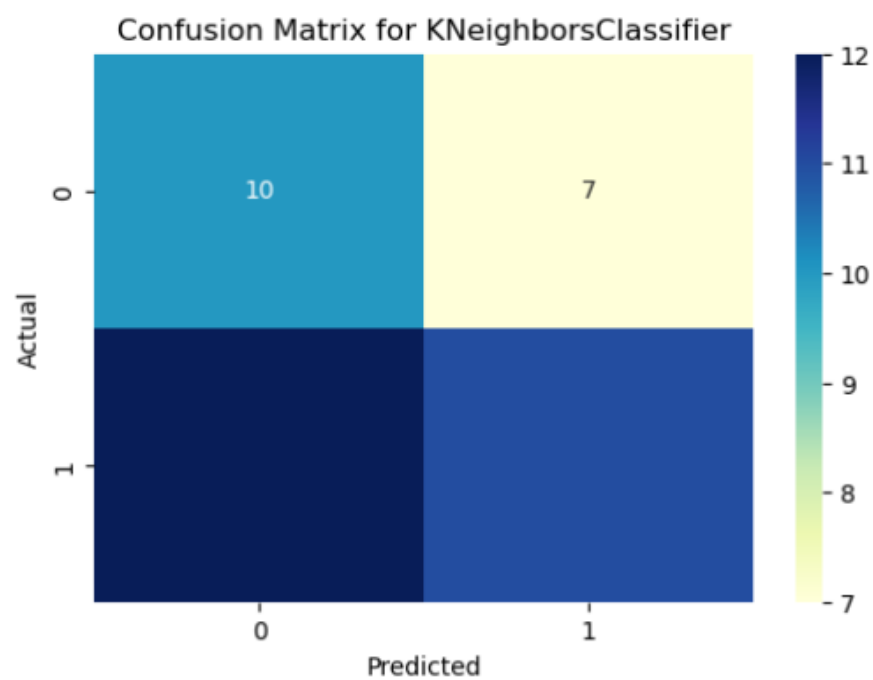
	precision	recall	f1-score	support
0	0.37	0.65	0.47	17
1	0.40	0.17	0.24	23
accuracy			0.38	40
macro avg	0.38	0.41	0.36	40
weighted avg	0.39	0.38	0.34	40

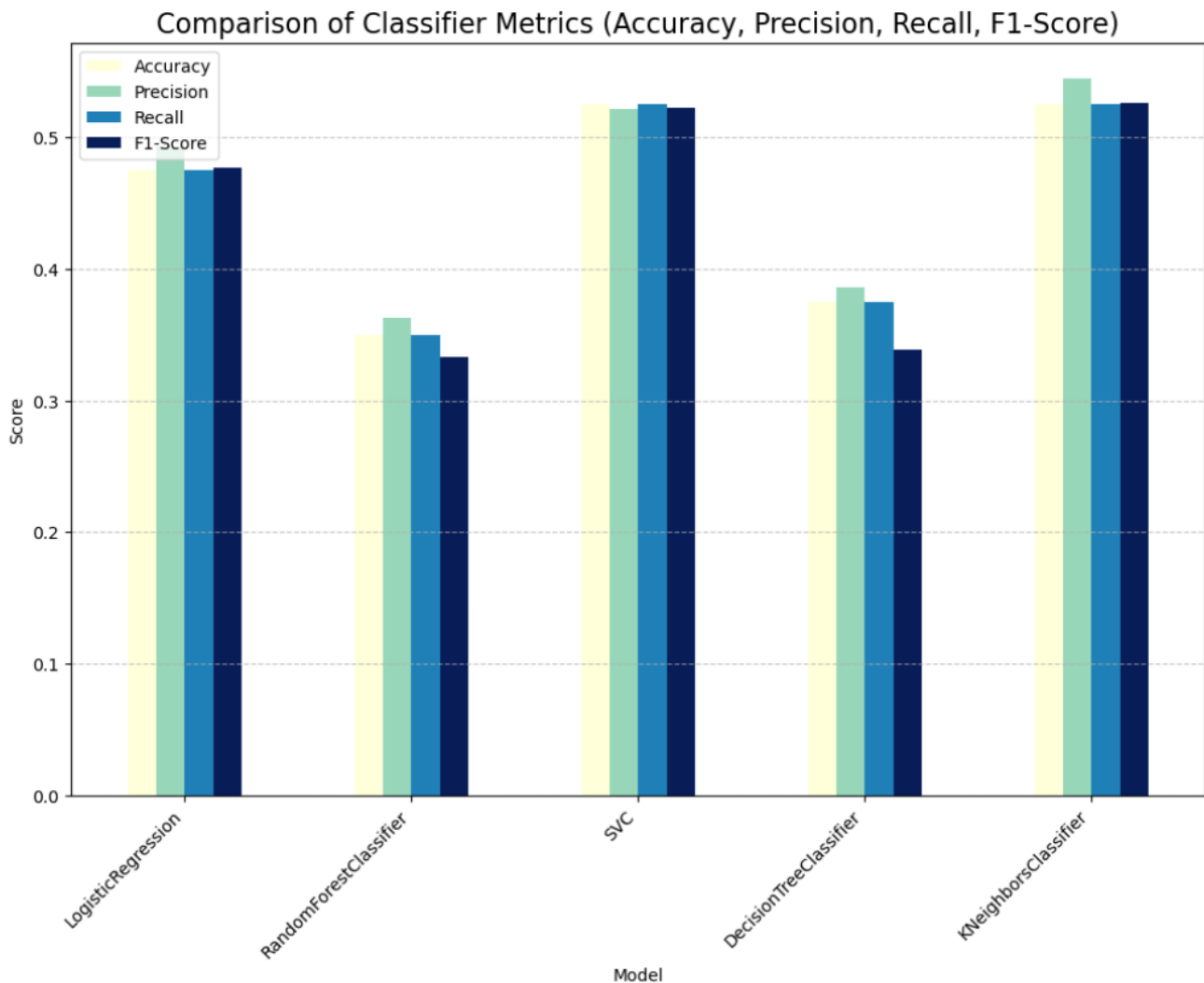


Evaluating K-Nearest Neighbors...

Classification Report for KNeighborsClassifier:

	precision	recall	f1-score	support
0	0.45	0.59	0.51	17
1	0.61	0.48	0.54	23
accuracy			0.53	40
macro avg	0.53	0.53	0.52	40
weighted avg	0.54	0.53	0.53	40





Conclusion

In this project, we applied multiple classification models (Logistic Regression, Random Forest, SVM, Decision Tree, and K-Nearest Neighbours) to predict employee attrition using a dataset containing demographic, job-related, and satisfaction metrics. After evaluating each model based on metrics like accuracy, precision, recall, and F1-score, we visualized their performance using confusion matrices and bar charts.

The analysis helps identify which factors contribute to employee attrition and highlights the best-performing models for predicting employee turnover. Insights gained from this analysis can guide HR departments in taking proactive measures to retain valuable employees, optimize job satisfaction, and reduce turnover rate.

